

World Development for Operation Code Rescue

Building a Complete, Novel, Release-Quality World

Operation Code Rescue Prepared with Claude

June 25, 2026

Contents

0.1	1. Introduction and Current State	2
0.2	2. World Architecture: World Partition, Data Layers, and OFPA	2
0.2.1	2.1 Why World Partition	2
0.2.2	2.2 Reconciling World Partition with 465 cities	3
0.2.3	2.3 Data Layers for state, time, and mode	3
0.2.4	2.4 Level Instances and Packed Level Actors: the reusable city kit	3
0.3	3. Procedural Content Generation (PCG) in UE 5.7	4
0.3.1	3.1 PCG is now production-ready	4
0.3.2	3.2 PCG as the natural successor to the C++ layers	4
0.3.3	3.3 Hybrid: hand-authored hero areas + PCG fill	4
0.4	4. Modular Environment Art and Kitbashing	5
0.4.1	4.1 A unique, art-directed survival-horror look per city family	5
0.4.2	4.2 Establishing the look	5
0.5	5. Lighting and Atmosphere	6
0.5.1	5.1 Lumen, VSM, and fog as the mood engine	6
0.5.2	5.2 Apple-Silicon / Metal caveats (critical for this project)	6
0.5.3	5.3 Lighting for readability and survival-horror mood	6
0.6	6. Level Design for <i>Operation Code Rescue</i>	7
0.6.1	6.1 Pacing: combat vs. coding “safe beats”	7
0.6.2	6.2 Sightlines, landmarks, and wayfinding across a city	7
0.6.3	6.3 Encounter spaces, the boss arena, and rescue set-pieces	7
0.6.4	6.4 Environmental storytelling for a coding-rescue narrative	7
0.7	7. Narrative and World-Building (Fiction)	8
0.7.1	7.1 A believable premise for “why coding rescues people”	8
0.7.2	7.2 A solo-scoped world bible	8
0.7.3	7.3 Conveying it through environment and audio	8
0.8	8. Scalability and Performance	8
0.8.1	8.1 Streaming budgets	8
0.8.2	8.2 HLODs and instancing	9
0.8.3	8.3 Author once, generate the rest	9
0.9	9. A Phased Roadmap (Solo-Scoped, with Validation Hooks)	9
0.9.1	Phase 0 — Foundations and guardrails (weeks)	9
0.9.2	Phase 1 — One authored kit + PCG, vertical slice (1–2 months)	9
0.9.3	Phase 2 — Authored + PCG hybrid systematized (2–4 months)	10
0.9.4	Phase 3 — Scaled campaign (ongoing)	10
0.9.5	Solo-dev scoping notes	10
0.10	10. Conclusion	10

0.1 1. Introduction and Current State

Operation Code Rescue is a first-person, post-apocalyptic survival-horror game in which the player solves real coding puzzles at in-world terminals while fighting zombies and rescuing survivors across a 465-city campaign — U.S. major cities at ranks 1–342 and global cities at 343–465 (Anchorage, Seattle, Tokyo, and so on). It is built solo in Unreal Engine 5.7 on macOS / Apple Silicon (Metal), with roughly 31,700 lines of C++.

The defining characteristic of the project’s world today is that **it has no authored .umap levels at all**. The entire world is generated from C++ at runtime in the engine entry map. The central class `ACodeRescueGameMode` exposes a `SpawnWorld()` routine and a long series of per-city layer builders — `SpawnCampaignCity()`, `SpawnUSCitySpecificIdentityLayer()`, `SpawnUSCityResidentialDistrictLayer()`, `SpawnUSCityVehiclePopulationLayer()`, `SpawnBespokeSurvivalLayer()`, `SpawnGameplayArenaConfinementLayer()`, and dozens more — each of which spawns blocks, Starter Content props, and Fab assets to approximate a city’s identity. Per-city data lives in the `FCodeRescueCityMission` struct (region, district style, landmark, art kit, accent colors, difficulty tier, skyline seed), and `FCodeRescueCampaign` holds the 465-entry catalog. A method tellingly named `EnsureCampaignCityLoaded()` is documented in the source as a “*procedural streaming stand-in for World Partition*.” Day/night is driven by a runtime-spawned `ADirectionalLight` (`SunLight`) plus a normalized `TimeOfDay`; weather, fog, sky, and grading are spawned from `UExponentialHeightFogComponent`, `ASkyAtmosphere`, `APostProcessVolume`, and Niagara systems — all instantiated in code rather than placed by an artist.

This is functional and impressively systemic, but it is not art-directed, not hand-crafted, and it cannot reach the variety and fidelity players expect from a commercial release. This chapter documents how to evolve — not discard — that runtime-procedural foundation into a professional **authored + procedural hybrid** that scales to hundreds of cities. The eight sections below map UE 5.7 systems onto the specific structure of *Operation Code Rescue* and end with a phased, solo-scoped roadmap with validation hooks.

A guiding principle throughout: **the existing C++ “city realization” layers are a data-driven recipe, not throwaway scaffolding**. The migration path is to keep the `FCodeRescueCityMission` data model and progressively replace the *block-spawning back end* with authored kits, Packed Level Actors, and PCG graphs.

0.2 2. World Architecture: World Partition, Data Layers, and OFPA

0.2.1 2.1 Why World Partition

World Partition stores the world in a single persistent Level and subdivides space into streamable grid cells loaded and unloaded at runtime by *streaming sources* (typically the player), so the engine only loads what the player can see or reach [1][2]. This is the production-grade replacement for the project’s hand-rolled `EnsureCampaignCityLoaded()` streaming stand-in. One File Per Actor

(OFPA) — enabled by default with World Partition — saves each actor to its own external file, removing the need to re-save the whole level when an actor changes [3]; this is also what makes solo iteration and any future version-control hygiene tractable.

0.2.2 2.2 Reconciling World Partition with 465 cities

A literal interpretation — one giant partitioned world containing 465 cities — is neither necessary nor wise. The campaign is structured as discrete, gated city missions with their own origins (`FCodeRescueCampaign::GetCityOrigin`), so the natural architecture is:

- **One World Partition map per *city family* (archetype), not per city.** The `FCodeRescueCityMission` already clusters cities by `DistrictStyle`, `RegionName`, `ArtKitName`, and home archetype (brownstone rows, painted Victorians, adobe ranches, deco pastels, mountain cabins, and so on). A dozen-or-so authored “kit worlds” can express the full catalog when combined with per-city data.
- **Per-city identity as data, applied on load.** Keep `FCodeRescueCityMission` as the source of truth. When a city is selected, the kit world streams in and a data-driven pass tints, reskins, and reconfigures it (accent colors, landmark, weather family, skyline seed) — exactly what the current `SpawnUSCitySpecificIdentityLayer()` family already does, but over authored geometry instead of cubes.

0.2.3 2.3 Data Layers for state, time, and mode

Data Layers organize actors both in-editor and at runtime, and can be dynamically activated or unloaded; layer assignments are stored per-actor in OFPA and a `WorldDataLayer` file [4]. For *Operation Code Rescue* this is the right tool for the project’s *existing axes of variation*:

- **Story/destruction state:** a “Pristine” vs. “Overrun” Data Layer per district lets the same kit show pre- and post-outbreak dressing.
- **Time-of-day / weather:** swap the project’s per-zone fog, sky, and grading into Data Layers (`Snow`, `MarineLayer`, `DesertClear`, `HumidGulf`) instead of spawning them imperatively in `ApplyUSCitySkyRealization()` and `SpawnWeatherForCity()`.
- **Game mode:** the project’s `bSandboxMode` (terminals only, no combat) maps cleanly to a Data Layer activation set, replacing branch-heavy spawn code.

0.2.4 2.4 Level Instances and Packed Level Actors: the reusable city kit

Level Instancing lets you build an assembly once and reuse it anywhere; **Packed Level Actors (PLAs)** reference a sub-level and bake its static-mesh content into a single optimized actor using Instanced/Hierarchical Instanced Static Mesh (ISM/HISM) components to collapse draw calls [5][6]. PLAs hold static meshes only — no skeletal meshes, particles, or dynamic actors — so they are ideal for buildings, walls, props, and street furniture, while the project’s dynamic content (zombies, survivors, terminals, the jeep, the boss) remains spawned as before [6].

This is the linchpin of the migration. The recommended modular hierarchy is: **trim-textured module meshes → wall/facade Packed Level Actors → building PLAs → block PLAs → district Level Instances**, with the player-facing city assembled by placing and tinting these

instances. Because PLA edits propagate to every instance [5], a single artist change updates the whole campaign — the productivity multiplier a solo developer needs.

0.3 3. Procedural Content Generation (PCG) in UE 5.7

0.3.1 3.1 PCG is now production-ready

The PCG Framework reached **Production-Ready** status in Unreal Engine 5.7 (Experimental in 2023, Beta in 5.4), with GPU and game-thread work pushing it to roughly twice the performance of 5.5 [7][8]. New in 5.7: a **PCG Editor Mode** offering spline-draw, point-paint, and volume tools, each backed by a PCG graph with live parameters and no code required; a **Polygon2D** data type for representing closed areas; new **Spline Intersection** / **Split Splines** operators; **standalone graph execution** (a graph can run without a world context or a PCG component); and an experimental **Procedural Vegetation Editor** for authoring Nanite foliage with Quixel Megaplants [7][8]. PCG also integrates directly with World Partition so generation respects streaming cells [9].

0.3.2 3.2 PCG as the natural successor to the C++ layers

The project’s spawn layers are already *rule-based content generation written in C++*. PCG is the same idea expressed in a visual, artist-tunable graph that the engine can run on the GPU and stream by cell. The migration is direct:

Current C++ approach	PCG equivalent in 5.7
<code>SpawnUSCityVehiclePopulationLayer()</code> placing curbside cars along a grid	PCG graph that samples the street-spline network and scatters a fleet-mix asset set with per-point density [7]
<code>SpawnUSCityLandscapeRealizationLayer()</code> ground tint + vegetation	PCG Biome Core graph driven by <code>BiomeDefinitions</code> / <code>BiomeAssets</code> / <code>BiomeGenerators</code> data assets [10][11]
<code>SpawnCinematicStreetLifeLayer()</code> clutter and debris	PCG scatter constrained by a Polygon2D district mask, with collision/overlap filtering [7]
<code>SkylineSeed</code> per <code>FCodeRescueCityMission</code>	PCG graph seed input, giving deterministic-but-varied blocks per city

The Biome Core sample is especially relevant: it demonstrates Attribute Set Tables, feedback loops, recursive subgraphs, and runtime hierarchical generation — placing assets on a 6400 cm grid via GPU HLSL nodes [10][11]. Cities’ regional vegetation families (evergreen ridge, palm shore, prairie horizon) map directly onto biome definitions.

0.3.3 3.3 Hybrid: hand-authored hero areas + PCG fill

The correct division of labor for a survival-horror world: **author the hero beats by hand, generate the connective tissue with PCG**. Hand-place the boss arena, each rescue set-piece, the safe-house interior, and the terminal “safe rooms” as Level Instances; let PCG populate the surrounding

blocks, abandoned-traffic snarls, foliage reclaiming the streets, and debris fields. PCG graphs should expose the same knobs the C++ already parameterizes (`AccentColor`, `density`, `DifficultyTier`) so per-city data continues to drive the look. Standalone graph execution [7] also lets PCG run as an offline asset-baking step, pre-generating block PLAs that then stream cheaply at runtime — the best of both authored and procedural worlds.

0.4 4. Modular Environment Art and Kitbashing

0.4.1 4.1 A unique, art-directed survival-horror look per city family

The path from “tinted cubes” to a commercial look is a disciplined modular kit per city archetype. The core techniques:

- **Modular building kits + trim sheets.** A trim sheet packs reusable surface details (brick courses, concrete trim, window mullions, signage strips) into one texture so many modular pieces share a single material; a handful of trim pieces can texture an entire wall without visible repetition [12]. This keeps texture memory flat as the city count grows.
- **Master material with vertex paint + tint.** A single master material driven by vertex painting, parallax/POM detail, and a color-tint parameter lets one mesh set produce wide variation [12]. This is precisely how the project should realize `AccentColor` / `SecondaryAccentColor` and the per-city wardrobe/grade tokens (`ActiveCityRealizationGradeToken`) — as material parameters, not new geometry.
- **Nanite for hero detail.** Nanite renders film-quality geometry without manual LODs and always renders to Virtual Shadow Maps regardless of distance as the most performant, highest-quality path [13][14]. Use Nanite for hero facades, rubble, and detail props; reserve traditional LODs for masked/translucent foliage where Nanite has caveats.
- **Megascans / Fab.** Quixel Megascans are now distributed through **Fab** (in-browser and in the Epic Games Launcher), the successor to Quixel Bridge, with one-click import to the Content Browser [15][16]. The project already ships several Fab zombie packs; the same pipeline supplies surfaces, decals, and hero props for environments.

0.4.2 4.2 Establishing the look

Survival-horror identity comes from restraint and consistency: a tight palette per city family, heavy use of grime/leak/rust decals, broken sightline geometry, and material storytelling (scorch marks, barricaded windows, evacuation notices). Build a **kit bible** per archetype defining the module set, trim layout, master-material parameters, and a “destruction dressing” pass. Because the project already encodes archetypes in `ArtKitName` / `DistrictStyle`, each kit bible maps 1:1 to existing data — the art simply replaces the block fallback in functions like `SpawnAuthoredPropsForCity()`, which is already written to *prefer* inspectable static-mesh props and fall back to blocks only when an asset is missing.

0.5 5. Lighting and Atmosphere

0.5.1 5.1 Lumen, VSM, and fog as the mood engine

- **Lumen** provides real-time global illumination and reflections and is the default GI for UE5 projects; it underpins believable bounce light and reflective wet streets central to horror mood.
- **Virtual Shadow Maps (VSMs)** deliver consistent high-resolution shadowing (a 16k×16k virtual resolution) designed to pair with Nanite, Lumen, and World Partition, replacing the patchwork of stationary-light techniques with one unified path; new projects use VSMs by default [13][14].
- **Exponential Height Fog + Volumetric Fog** are the project’s existing atmosphere tools (it already spawns `UExponentialHeightFogComponent`). Volumetric fog catching light shafts is the single highest-impact lever for survival-horror readability and dread.

0.5.2 5.2 Apple-Silicon / Metal caveats (critical for this project)

Lumen on Metal supports both software and hardware ray tracing, but capability is generation-dependent: **software ray tracing runs on M1/M2/M3; hardware ray tracing targets M3 and newer and is still maturing** [17]. On recent high-end parts (e.g., M5 Max), hardware RT lands around the mid-tier discrete-GPU bracket — workable for production iteration but not a substitute for a high-end NVIDIA card on the heaviest scenes; the Metal path tracer is functional but slower than the equivalent D3D12 path, so final-quality path-traced cinematics belong on Windows + NVIDIA [17]. Epic has steadily closed the macOS/Windows feature gap, but parity is not absolute [18][19]. Practical guidance for this solo/Metal project:

1. **Default to Lumen software ray tracing** for development and the shipping macOS target; treat hardware RT as an opt-in quality tier gated on chip generation.
2. **Budget VSM and Lumen aggressively** — cap Lumen reflection quality, lean on screen-space where acceptable, and validate frame time on the actual development Mac, not a Windows reference.
3. **Keep the day/night system, but render it through proper sky.** The project already drives a `ADirectionalLight` sun and `ASkyAtmosphere`; pairing those with VSM-cast moving shadows and time-graded post volumes is exactly the readability-vs-dread balance horror needs — bright enough to navigate at the terminal “safe beats,” oppressively dark on the approach.

0.5.3 5.3 Lighting for readability and survival-horror mood

Use lighting as a *wayfinding and pacing tool*, not just decoration: warm, higher-key pools of light at safe beats (terminals, safehouse, extraction helipad); cold, low-key, high-contrast lighting on combat approaches; a single readable key direction so the player can always parse threat silhouettes. Drive these per-beat looks with the project’s existing per-zone post-process volumes (`SpawnPerZonePostProcessVolume()`) and grade tokens rather than ad-hoc light placement.

0.6 6. Level Design for *Operation Code Rescue*

0.6.1 6.1 Pacing: combat vs. coding “safe beats”

The game’s core rhythm is a deliberate oscillation between **tension** (combat/traversal) and **focus** (coding at a terminal). Solving a coding puzzle requires the player to stop and read; the level must make that physically safe and psychologically calm. Design terminals as defensible **safe rooms**: limited entrances the player can barricade (the project has `ABarricadeActor`), clear sightlines to approaches, warm light, and an audible “all clear.” The existing `TriggerBossHorde()` (a 30-second rush spawned around a just-solved terminal) is a strong beat — but it should be a *deliberate, telegraphed* spike at the end of a calm window, not constant pressure that prevents reading code.

0.6.2 6.2 Sightlines, landmarks, and wayfinding across a city

A readable city needs a legible mental map. Apply Lynch-style wayfinding: strong **landmarks** (the project already assigns a `LandmarkName` per city), clear **paths** (the authored street grid the vehicle/route layers assume), and distinct **districts** (`DistrictStyle`). Use silhouette and lighting to point the player toward the next objective; the project’s `SpawnMissionObjectiveRoute()` becomes a designed critical path with intentional vistas rather than a debug line. The current `SpawnGameplayArenaConfinementLayer()` (soft-bounding the playable area) should be reframed as designed level boundaries — collapsed overpasses, debris walls, flooded streets — that read as world, not invisible walls.

0.6.3 6.3 Encounter spaces, the boss arena, and rescue set-pieces

- **Encounter spaces** should vary cover density and escape routes to create distinct combat textures; tie spawn intensity to the city’s `EncounterIntensity` and `DifficultyTier`.
- **The boss arena** (`SpawnBossForCity()`, placed deep in a quadrant so the player fights inward) deserves bespoke authoring: a defined silhouette, arena geometry that supports the boss’s mechanics, and lighting that frames the fight. This is a hand-authored Level Instance, not procedural.
- **Rescue set-pieces** (`SpawnSurvivorReliefCamp()`, `SpawnEnterableCivicSafehouse()`, the helipad extraction) are the emotional payoffs — author them by hand with environmental storytelling and stage them so rescue and extraction are spatially clear.

0.6.4 6.4 Environmental storytelling for a coding-rescue narrative

Environmental storytelling uses physical space, props, and lighting to convey narrative without dialogue, and is especially powerful in post-apocalyptic worlds [20][21]. For this game the through-line is unique: tell the story of *a society that ran on code and is being recovered through code*. Dress the world with the fiction — dead server racks, hand-written sticky notes on dark monitors, evacuation routes printed from a failing system, terminals as fragile islands of working technology. Each terminal’s puzzle should feel like it *does something* in the fiction (reopening a door, restoring power, decrypting a survivor’s location), reinforced by the surrounding set dressing.

0.7 7. Narrative and World-Building (Fiction)

0.7.1 7.1 A believable premise for “why coding rescues people”

The premise must make coding *diegetically* load-bearing rather than a minigame bolted on. A coherent, novel framing: in the world of *Operation Code Rescue*, near-total automation meant infrastructure — power grids, transit, medical systems, the locks on every door — ran on networked code. When the outbreak corrupted or orphaned those systems, survivors became trapped behind machines that no longer had anyone to operate them. The player is a field engineer who restores or rewrites the code that controls the physical world: every solved terminal literally reopens a path, powers a safehouse, or pinpoints a survivor. This makes the player’s real coding skill the narrative’s source of agency — the puzzles *are* the rescues.

0.7.2 7.2 A solo-scoped world bible

A world bible captures geography, history, factions, technology, and tone, and exists so all environment, audio, and writing decisions stay consistent [22]. For a solo developer, keep it lean but real:

- **Pillars:** survival-horror dread; coding-as-empowerment; a 465-city journey from local (Anchorage/Seattle) to global (Tokyo and beyond) as a widening rescue effort.
- **Factions / forces (light touch):** the surviving engineers’ network (the player’s allies — already echoed by the Engineer/Medic/Scientist/Trader friendly NPCs the project spawns), the abandoned automated systems themselves as antagonistic environment, and the infected.
- **Technology rules:** what code can and cannot fix, so puzzles feel fair and grounded.
- **Per-city lore as data:** the `MissionBrief`, `RadioBriefing`, `CharacterStoryPlan`, and `SurvivorName` fields in `FCodeRescueCityMission` are already a per-city lore database — formalize them against the bible so each city contributes a consistent chapter.

0.7.3 7.3 Conveying it through environment and audio

The project already has **230 voiced radio briefings** plus a system-TTS narrator (`SpeakRadioBriefing()`, `CityRadioBriefingCues`). Audio is the cheapest, highest-bandwidth lore channel for a solo developer: the radio frames each city’s situation on arrival, ambient zone beds (`ZoneAmbientCues`) set mood, and environmental dressing fills in detail without cutscenes. The discipline is to **show, not tell** [22] — let the radio raise a question that the environment answers as the player moves through the city.

0.8 8. Scalability and Performance

0.8.1 8.1 Streaming budgets

World Partition exposes per-frame streaming budgets to prevent hitches — limiting cells loaded per frame (`wp.Runtime.MaxStreamingCellsPerFrame`), actors spawned per frame (`wp.Runtime.MaxActorsToSpawnPerFrame`), and a memory budget (`wp.Runtime.MemoryBudgetMB`) [2]. These replace the project’s manual `ClearStreamedCampaignActors()` / `RegisterStreamedActor()`

bookkeeping with engine-managed streaming, and `wp.Runtime.ToggleDrawRuntimeHash2D` visualizes loaded cells at runtime for tuning [23].

0.8.2 8.2 HLODs and instancing

HLODs generate a single proxy mesh + material for distant unloaded cells, drastically cutting draw calls — essential for a city skyline visible beyond the streamed cells [23]. HLOD Layers support three layer types: **Instancing** (ISM at lowest LOD, ideal for trees/foilage), **Merged Mesh**, and **Simplified Mesh**, and can output Nanite-enabled proxies [23]. Actors must be Static and assigned to an HLOD Layer; build via **Build > Build HLODs** or the `WorldPartitionHLODsBuilder` commandlet, budgeting tens of minutes to hours for large worlds [23]. Combined with the PLA/ISM kit from §2.4, this is how a dense city renders within a solo-feasible draw-call budget.

0.8.3 8.3 Author once, generate the rest

The scalability thesis for 465 cities solo:

- **Author once (10–15 kits):** module meshes, trim sheets, master materials, building/block PLAs, hero set-pieces (boss arena, safehouse, rescue camp templates), and one PCG graph set per biome/city family.
- **Generate / data-drive everything else:** block layout, foliage, traffic, debris, and per-city tinting/landmark/weather via PCG graphs and the existing `FCodeRescueCityMission` data.

This preserves the project’s greatest asset — a fully data-driven 465-city catalog — while concentrating hand-craft where it is seen most.

0.9 9. A Phased Roadmap (Solo-Scoped, with Validation Hooks)

0.9.1 Phase 0 — Foundations and guardrails (weeks)

- Stand up **Data Validation**: UE ships the Data Validation plugin enabled by default; author `UEditorValidatorBase` subclasses (C++) to enforce naming, missing-asset, and performance-budget rules, runnable from the editor’s Tools menu [24][25]. Wire the project’s existing automated world/visual checks into this framework so every kit and city passes the same gate.
- Convert the entry map to **World Partition** with OFPA; introduce baseline **Data Layers** (Pristine/Overrun, weather, sandbox) [1][3][4].

0.9.2 Phase 1 — One authored kit + PCG, vertical slice (1–2 months)

- Build a single **city-family kit** (modules, trim sheets, master material) and assemble one hero city as Level Instances / PLAs, replacing block fallbacks in that city only [5][6][12].
- Add **one PCG graph set** (block scatter, foliage via Biome Core, traffic) driven by `FCodeRescueCityMission` data [7][10].
- Light it with Lumen (software RT) + VSM + volumetric fog; validate frame time **on the development Mac** [13][17].

- **QA hook:** the vertical-slice city must pass Data Validation and the project’s visual checks, and hit a fixed frame-time budget on Apple Silicon.

0.9.3 Phase 2 — Authored + PCG hybrid systematized (2–4 months)

- Generalize the kit into the **archetype set** (10–15 kits) mapped to `ArtKitName` / `DistrictStyle`.
- Move per-city sky/weather/grade from imperative spawns into **Data Layers + material parameters**.
- Hand-author the **boss arena, safehouse, and rescue set-piece** templates as reusable Level Instances.
- Generate **HLODs** for distant skylines via the commandlet; set **streaming budgets** [2][23].

0.9.4 Phase 3 — Scaled campaign (ongoing)

- Drive all 465 cities through kit + PCG + data; spot-author signature global landmarks (Tokyo, etc.) as hero overrides.
- Establish an **offline PCG bake step** (standalone graph execution) to pre-generate block PLAs for streaming efficiency [7].
- **QA hook:** an automated pass that loads each city, runs Data Validation, captures a reference screenshot, and checks streaming/memory budgets — the scaled version of the project’s existing per-city automated checks.

0.9.5 Solo-dev scoping notes

Concentrate hand-authoring on the 10–15 kits and the handful of hero set-pieces; let data and PCG carry breadth. Treat the existing C++ spawn layers as the *fallback and the spec* — keep them working so the game is always shippable, and retire each one only as its authored/PCG replacement passes validation. This guarantees continuous playability while quality ratchets upward city by city.

0.10 10. Conclusion

Operation Code Rescue already has the hardest part of a scalable world: a complete, data-driven, 465-city model with per-city identity, pacing systems, day/night, weather, and a strong diegetic hook for coding-as-rescue. What it lacks is authored fidelity. The migration is therefore additive, not destructive: adopt World Partition + Data Layers + OFPA for streaming and state; build a modular PLA kit per city family with trim-sheet materials and Nanite hero detail; let PCG (now production-ready in 5.7) generate the connective tissue the C++ layers currently fake; light for survival-horror mood within Apple-Silicon Lumen budgets; hand-author only the hero beats; and gate everything behind Data Validation and automated per-city checks. Executed in phases, this turns a clever runtime prototype into a commercial-grade, novel world that a solo developer can realistically scale to hundreds of cities.

0.11 References

- [1] World Partition in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/en-us/unreal-engine/world-partition-in-unreal-engine> [2] UE5 World Partition Tutorial: Open World Streaming (streaming budgets) — <https://altheragames.com/en/blog/ue5-world-partition-guide> [3] One File Per Actor in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/unreal-engine/one-file-per-actor-in-unreal-engine?lang=en-US> [4] World Partition - Data Layers in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/en-us/unreal-engine/world-partition---data-layers-in-unreal-engine> [5] A Quick Guide: Packed Level Actors & Level Instancing in Unreal Engine — <https://help.kitbash3d.com/en/articles/12038349-a-quick-guide-packed-level-actors-level-instancing-in-unreal-engine-with-kitbash3d> [6] Level Instancing in Unreal Engine | UE Documentation — <https://dev.epicgames.com/documentation/unreal-engine/level-instancing-in-unreal-engine?lang=en-US> [7] The Unreal Engine 5.7 procedural content generation update explained | Creative Bloq — <https://www.creativebloq.com/3d/video-game-design/the-unreal-engine-5-7-procedural-content-generation-update-explained> [8] Unreal Engine 5.7 Released With New Procedural Content Generation and More Features | TechPowerUp — <https://www.techpowerup.com/342868/unreal-engine-57-released-with-new-procedural-content-generation-and-more-features> [9] Using PCG with World Partition in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/unreal-engine/using-pcg-with-world-partition-in-unreal-engine?lang=en-US> [10] PCG Biome Core and Sample Plugins Reference Guide | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-pcg-biome-core-and-sample-plugins-reference-guide-in-unreal-engine> [11] PCG Biome Core and Sample Plugins Overview Guide | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/en-us/unreal-engine/procedural-content-generation-pcg-biome-core-and-sample-plugins-overview-guide-in-unreal-engine> [12] Best practices creating Nanite modular environments — Epic Developer Community Forums — <https://forums.unrealengine.com/t/best-practices-creating-nanite-modular-environments/768574> [13] Virtual Shadow Maps in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/en-us/unreal-engine/virtual-shadow-maps-in-unreal-engine> [14] Nanite Technical Details | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/unreal-engine/nanite-technical-details> [15] Fab in Launcher brings Quixel Bridge features to the Epic Games Launcher | Quixel — <https://quixel.com/news/fab-in-launcher-brings-quixel-bridge-features-to-the-epic-games-launcher> [16] Quixel to Fab Transition FAQs | Fab Support — https://support.fab.com/s/article/Fab-Transition-FAQs?language=en_US [17] Apple Silicon M5 for Unreal Engine Development: Viability in 2026 | StraySpark — <https://www.strayspark.studio/blog/apple-silicon-m5-unreal-engine-development-2026> [18] Bringing Unreal Engine on macOS up to feature parity with Windows — progress report | Unreal Engine — <https://www.unrealengine.com/tech-blog/bringing-unreal-engine-on-macos-up-to-feature-parity-with-windowsprogress-report> [19] Unreal Engine 5.2 brings native support for Apple Silicon | Unreal Engine — <https://www.unrealengine.com/en-US/tech-blog/unreal-engine-5-2-brings-native-support-for-apple-silicon-and-other-developments-for-macos> [20] Environmental Storytelling | Game Developer — <https://www.gamedeveloper.com/design/environmental-storytelling> [21] Horror Survival Level Design: Environment and Story | World of Level Design — https://www.worldofleveldesign.com/categories/level_design_tutorials/horror-fear-level

[design/part3-survival-horror-level-design-story-environment.php](#) [22] Worldbuilding in Game Development: Meaning, Elements, Steps, Tips | Game Design Skills — <https://gamedesignskills.com/game-design/worldbuilding/> [23] World Partition - Hierarchical Level of Detail in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/en-us/unreal-engine/world-partition---hierarchical-level-of-detail-in-unreal-engine> [24] Data Validation in Unreal Engine | UE 5.7 Documentation — <https://dev.epicgames.com/documentation/unreal-engine/data-validation-in-unreal-engine> [25] Unreal Asset Validation | SteveStreeting.com — <https://www.stevestreeting.com/2024/12/17/unreal-asset-validation/>